

02: Context, @ Mentions, and Modes (Build & Plan)

Introduction

This section explains three pieces of opencode's core interaction model:

- **@ mentions** — how you tell opencode exactly which files (or subagents) to look at.
- **Build mode** — opencode's default primary agent, with full read/edit/run access.
- **Plan mode** — a restricted, analysis-only primary agent for reviewing an approach before anything is changed.

No exercises here — this is a reference for the concepts. Hands-on practice with all three lives in [hands-on/](#).

@ Mentions

`opencode` does not automatically hold your whole repository in its context window. The `@` key is how you target what exactly to look at, instead of hoping it finds the right file on its own.

- Type `@` followed by part of a filename (e.g. `@InventoryItem`), and opencode fuzzy-searches the project, showing matching files. Arrow keys + Enter (or Tab) select one and insert it as a file reference, e.g.:

```
@src/main/java/com/atlas/inventory/InventoryItem.java What does the quote() helper method do, and why does it exist?
```

- You can mention **more than one file** in the same prompt when you need opencode to compare or cross-reference them:

```
Compare @src/main/java/com/atlas/inventory/InventoryItem.java and  
@src/main/java/com/atlas/inventory/InventoryServer.java. Both files contain  
JSON string-escaping logic. Are they identical? If not, what's different?
```

- The same `@` key also addresses a **subagent** directly, instead of waiting for Build mode to decide whether to delegate. Typing `@` on its own shows built-in subagents (such as `explore`) alongside file suggestions:

```
@explore where in this codebase is a duplicate part number rejected, and with  
what HTTP status code?
```

`explore` is read-only — it can search and read files but cannot edit them, which makes it safe to reach for whenever you want an answer without any risk of a side effect. (`opencode` also ships a `general` subagent, run `opencode agent list` in the CLI to see what your install provides.)

`opencode` also ships a handful of built-in slash commands, separate from anything you configure yourself (note that there is a difference between the commands available through the Desktop app and the CLI tool):

- `/undo` — reverts the agent's last change and brings your prompt back so you can try again.
- `/redo` — brings a reverted change back.
- `/share` — creates a link to the current conversation (read-only). Treat it like posting a chat transcript, because that's exactly what it is.
- `/help` — lists available commands.

Build mode (Agent mode)

Build is `opencode`'s default primary agent. It has full tool access: it can read, search, edit files, and run commands. This is where you implement complete, small changes. Build mode is also useful for small, everyday refactors on code you point it at directly.

Typical flow:

1. reference the function or file with `@`
2. describe the change
3. review the diff before approving each edit.

Check the mode indicator in the lower-right corner of the `opencode` TUI to confirm you're in Build; press `Tab` to switch modes (use tab plus arrow keys in the OpenCode Desktop app).

Plan mode

Plan is `opencode`'s restricted, analysis-only primary agent. It cannot edit your code at all: Plan carries a built-in `edit` → `deny` rule for every path, with a narrow exception for plan files under `.opencode/plans/`. Bash is not restricted by the agent itself — shell commands still go through this

repo's `opencode.json` rules, which default to `ask`.

So Plan can read, search, and reason freely, and it will ask before running a command, but an edit to `InventoryService.java` is refused outright rather than offered to you for approval. Note that the project's `"edit": "ask"` setting does **not** loosen this — the agent's own rule overrules.

Check what your install actually does rather than trusting this page:

```
opencode agent list
```

Look at the `plan (primary)` block's `edit` entries. opencode's published docs have described Plan's edit permission as `ask`; the binary shipped as of 1.18.20 denies it. If your version disagrees with this file, believe `opencode agent list`.

Typical flow:

1. Press `Tab` to switch to **Plan**.
2. Describe the feature or change you want planned. Plan mode may ask clarifying questions — answer them.
3. Read the plan fully. Push back on anything unclear or wrong, and ask for a revision — Plan mode will not sneak in an edit while you're still evaluating the approach.
4. Once satisfied, either:
 - Press `Tab` to switch back to **Build** and hand the plan off for implementation, or
 - Ask opencode to save the plan to a file instead of implementing it, so a bigger or riskier change can be reviewed by the rest of the team before anyone touches a line of code. Use a path under `.opencode/plans/` (e.g. `.opencode/plans/summary-refactor.md`), as that is the one place Plan mode is allowed to write to. Asking it for `docs/some-plan.md` while still in Plan mode will be refused. Switch to Build mode first if you want the plan to live somewhere else.

Switching modes is a trivial skill. The judgment call of when to hand a plan over to Build versus stopping and saving it for review is the real competence that Plan mode teaches.

Where to practice this

Hands-on exercises covering context, mentions, Build mode, and Plan mode together are in [hands-on/](#).